

# MEMORIA PROYECTO IoT

Sistema IoT para la monitorización de plantas mediante una Raspberry Pi Pico W, un sensor de humedad y una pantalla TFT. El dispositivo permite supervisar el estado de la planta en tiempo real, representarlo mediante un Tamagotchi interactivo y almacenar los datos para su posterior consulta y visualización.

## CONTEXTO DEL PROYECTO

El cuidado adecuado de las plantas requiere mantener unas condiciones de humedad apropiadas para garantizar su crecimiento y conservación. Sin embargo, en muchas ocasiones los usuarios desconocen cuándo es necesario regar o no disponen de información suficiente para evaluar el estado real de la planta, lo que puede provocar problemas derivados tanto de la falta como del exceso de agua.

Con el avance de las tecnologías de monitorización y comunicación, es posible desarrollar sistemas capaces de supervisar automáticamente diferentes variables ambientales y proporcionar información en tiempo real al usuario. Estas soluciones permiten realizar un seguimiento continuo del estado de una planta y facilitan la detección temprana de situaciones que puedan afectar a su salud.

En este contexto, se plantea el desarrollo de un sistema de monitorización de plantas que permita medir la humedad del suelo, visualizar el estado de la planta mediante una interfaz gráfica basada en un Tamagotchi y consultar la información recopilada a través de distintas herramientas de visualización y notificación.

## OBJETIVOS PERSEGUIDOS

Los principales objetivos del proyecto son:

- Diseñar e implementar un dispositivo IoT capaz de monitorizar el nivel de humedad de una planta en tiempo real.
- Utilizar la conectividad WiFi de la Raspberry Pi Pico W para transmitir datos mediante el protocolo MQTT.
- Almacenar las mediciones obtenidas para permitir su análisis histórico.
- Visualizar la información mediante herramientas de monitorización y paneles gráficos con Grafana.
- Desarrollar una interfaz visual basada en un Tamagotchi que refleje el estado de la planta de forma intuitiva para el usuario.
- Integrar un sistema de notificaciones y consulta remota mediante Telegram.

- Aplicar los conocimientos adquiridos en la asignatura sobre sensores, protocolos IoT, comunicaciones y visualización de datos.

El resultado final es una solución de bajo coste capaz de supervisar el estado de una planta, ofrecer información en tiempo real al usuario y proporcionar una representación visual atractiva que mejore la interacción con el sistema.

## TRABAJO Y METODOLOGÍA

El desarrollo del proyecto se ha realizado siguiendo una metodología incremental, implementando y validando cada componente de forma independiente antes de integrarlo con el resto del sistema.

En una primera fase se estudió la viabilidad del proyecto y se seleccionaron los componentes hardware necesarios, concretamente la Raspberry Pi Pico W, el sensor capacitivo de humedad del suelo y la pantalla TFT ST7735S.

Posteriormente se desarrolló el firmware de la Raspberry Pi Pico W, encargado de la lectura del sensor de humedad, la conexión a la red WiFi y el envío periódico de datos mediante el protocolo MQTT.

Una vez establecida la comunicación, se implementó el sistema de almacenamiento utilizando Mosquitto como broker MQTT e InfluxDB como base de datos temporal para registrar las medidas obtenidas por el sensor.

A continuación, se desarrolló un dashboard en Grafana para visualizar la evolución de la humedad del suelo en tiempo real y consultar el histórico de medidas registradas.

Paralelamente se diseñó la interfaz visual del proyecto mediante un personaje tipo Tamagotchi mostrado en la pantalla TFT. Se crearon diferentes estados visuales asociados a distintos niveles de humedad para representar de forma intuitiva el estado de la planta.

Finalmente se desarrolló un bot de Telegram que permite consultar remotamente la última medición disponible y el estado actual de la planta.

Durante todo el desarrollo se utilizó Git y GitHub para el control de versiones y el trabajo colaborativo, permitiendo mantener un seguimiento de los cambios realizados y facilitar la integración del trabajo de ambas integrantes del equipo.

# DISEÑO DEL SISTEMA

## 1 Arquitectura

El sistema desarrollado sigue una arquitectura IoT distribuida compuesta por cuatro bloques principales: adquisición de datos, comunicación, almacenamiento y visualización.

La adquisición de datos se realiza mediante un sensor capacitivo de humedad conectado a una Raspberry Pi Pico W. El dispositivo realiza lecturas periódicas de la humedad del suelo y las procesa para obtener un porcentaje de humedad comprensible para el usuario.

Una vez obtenidos los datos, la Raspberry Pi Pico W los transmite a través de la red WiFi utilizando el protocolo MQTT. Los mensajes son enviados al broker Mosquitto, que actúa como intermediario entre el dispositivo IoT y los servicios de almacenamiento y monitorización.

Los datos recibidos son procesados por un script Python encargado de suscribirse al tópico MQTT y almacenarlos en una base de datos InfluxDB. Esta base de datos permite conservar un histórico de las medidas y facilita su posterior consulta.

Finalmente, la información almacenada puede visualizarse mediante Grafana a través de un dashboard interactivo y también puede consultarse remotamente mediante un bot de Telegram. De forma paralela, la Raspberry Pi Pico W muestra en la pantalla TFT una representación visual del estado de la planta mediante un personaje tipo Tamagotchi.

Pequeño esquema de funcionamiento:

Sensor

↓

Pico W

↓ MQTT

Mosquitto

↓

InfluxDB

↙ ↘

Grafana Telegram

## 2 Hardware

El hardware utilizado en el proyecto está compuesto por los siguientes elementos:

### Raspberry Pi Pico W

La Raspberry Pi Pico W constituye el núcleo del sistema. Es responsable de la lectura del sensor, la conexión a la red WiFi y el envío de los datos mediante MQTT.

### Capacitive Soil Moisture Sensor V1.2

Se ha utilizado un sensor capacitivo de humedad del suelo para medir el nivel de agua presente en la tierra. Este tipo de sensor presenta una mayor durabilidad que los sensores resistivos convencionales, ya que evita problemas de corrosión producidos por el contacto continuo con la humedad.

El sensor proporciona una señal analógica que es convertida a porcentaje de humedad mediante una calibración previa realizada durante el desarrollo del proyecto.

### Pantalla TFT ST7735S

La interfaz visual del sistema se implementa mediante una pantalla TFT ST7735S a color con resolución 128x160 píxeles.

La pantalla muestra diferentes imágenes asociadas a los distintos estados de la planta, permitiendo conocer visualmente su situación sin necesidad de consultar herramientas externas.

| Componente          | Función          |
|---------------------|------------------|
| Raspberry Pi Pico W | Procesamiento    |
| Sensor              | Medición humedad |
| Pantalla            | Interfaz         |
| WiFi                | Comunicación     |

## 3 Software

El software desarrollado se divide en dos bloques principales: el software embebido ejecutado en la Raspberry Pi Pico W y el software de servidor ejecutado en un ordenador.

### Software embebido

El firmware ha sido desarrollado en MicroPython y está formado por varios módulos:

- `config.py`: almacena los parámetros de configuración del sistema.
- `sensores.py`: realiza la lectura y procesamiento de la humedad del suelo.
- `main.py`: coordina el funcionamiento general del dispositivo.
- `oled_display.py`: controla la visualización de imágenes en la pantalla TFT.

El dispositivo realiza lecturas periódicas del sensor, determina el estado de la planta y publica los datos mediante MQTT.

### Software de servidor

El servidor ha sido desarrollado en Python y está compuesto por:

- `mqtt_to_influx.py`: recibe los mensajes MQTT y almacena las medidas en InfluxDB.
- `bot_telegram.py`: permite consultar remotamente el estado de la planta mediante Telegram.

### Base de datos

Se utiliza InfluxDB como sistema de almacenamiento temporal para registrar las medidas de humedad obtenidas por el sensor.

### Visualización

Grafana se emplea para representar gráficamente la evolución de la humedad del suelo y consultar el histórico de medidas registradas.

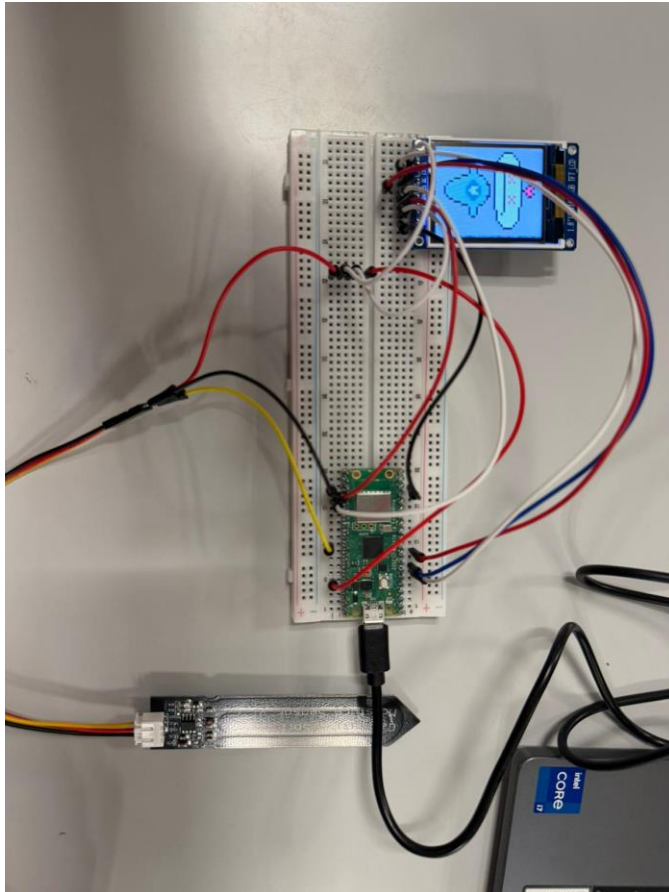
### Control de versiones

El desarrollo se ha gestionado mediante Git y GitHub, facilitando el trabajo colaborativo y el seguimiento de los cambios realizados durante el proyecto.

## RESULTADOS OBTENIDOS

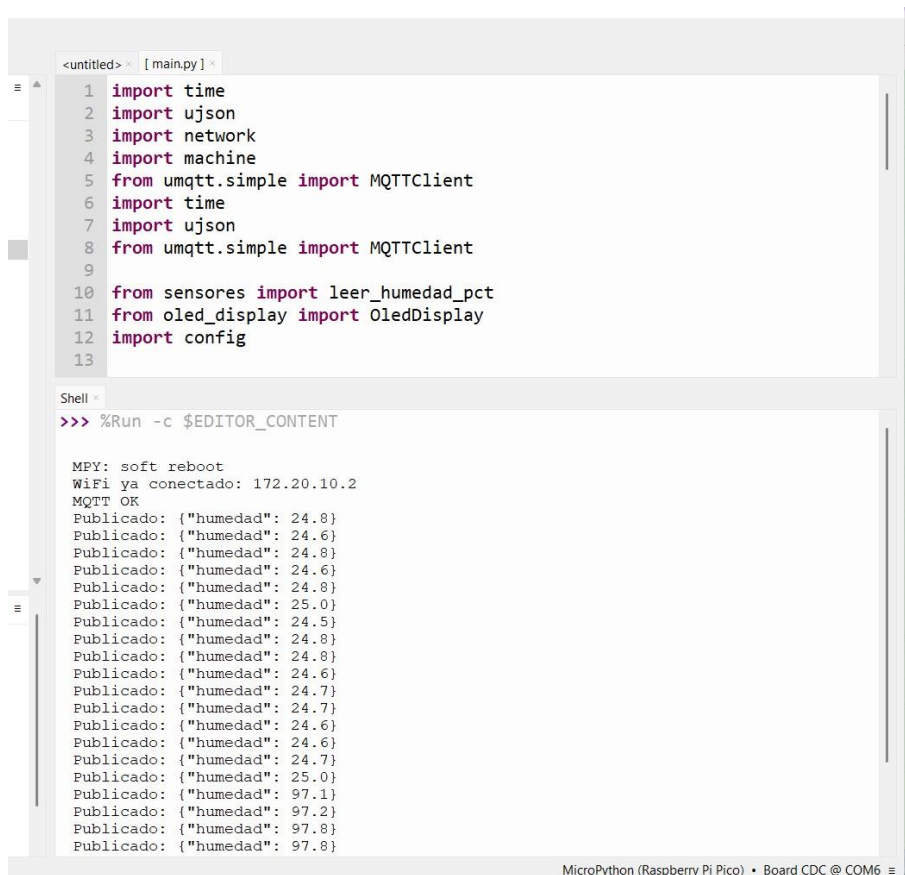
Una vez finalizado el desarrollo del proyecto, se obtuvo un sistema IoT completamente funcional capaz de monitorizar el estado de una planta en tiempo real. El sistema integra todos los componentes diseñados durante el desarrollo, incluyendo la Raspberry Pi Pico W, el sensor capacitivo de humedad, la pantalla TFT, el broker MQTT, la base de datos InfluxDB, Grafana y el bot de Telegram.

La **Figura 1** muestra el montaje final del proyecto, donde pueden observarse los principales componentes hardware empleados y su integración para realizar la monitorización de la planta.



En primer lugar, se validó el funcionamiento de la Raspberry Pi Pico W. La placa establece correctamente la conexión a la red WiFi y al broker MQTT, realizando lecturas periódicas del sensor de humedad y publicando dichas medidas en el tópico configurado. Como puede observarse en la **Figura 2**, cada lectura es convertida a un porcentaje de humedad y enviada automáticamente mediante MQTT.

**Figura 2.** Ejecución del programa principal en la Raspberry Pi Pico W y publicación periódica de las medidas de humedad.



The screenshot shows a MicroPython IDE window with a file named `main.py`. The code in the editor includes imports for `time`, `ujson`, `network`, `machine`, `MQTTClient`, `oled_display`, and `config`. It also includes imports for `leer_humedad_pct` and `OledDisplay`. Below the code editor is a shell window showing the execution of the script. The output indicates a soft reboot, successful WiFi connection to 172.20.10.2, and MQTT connection success. It then shows a series of messages being published to the MQTT broker, each containing a humidity reading (e.g., `Publicado: {"humedad": 24.8}`).

```
<untitled> x [main.py] x
1 import time
2 import ujson
3 import network
4 import machine
5 from umqtt.simple import MQTTClient
6 import time
7 import ujson
8 from umqtt.simple import MQTTClient
9
10 from sensores import leer_humedad_pct
11 from oled_display import OledDisplay
12 import config
13

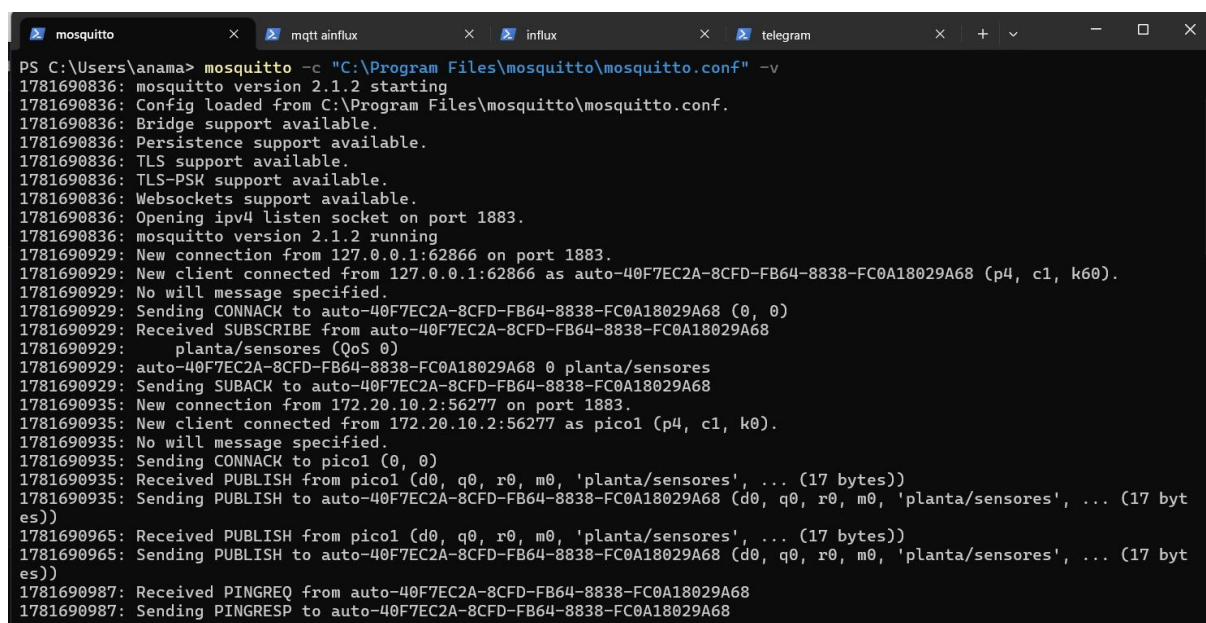
Shell x
>>> %Run -c $EDITOR_CONTENT

MPY: soft reboot
WiFi ya conectado: 172.20.10.2
MQTT OK
Publicado: {"humedad": 24.8}
Publicado: {"humedad": 24.6}
Publicado: {"humedad": 24.8}
Publicado: {"humedad": 24.6}
Publicado: {"humedad": 24.8}
Publicado: {"humedad": 25.0}
Publicado: {"humedad": 24.5}
Publicado: {"humedad": 24.8}
Publicado: {"humedad": 24.8}
Publicado: {"humedad": 24.6}
Publicado: {"humedad": 24.7}
Publicado: {"humedad": 24.7}
Publicado: {"humedad": 24.6}
Publicado: {"humedad": 24.6}
Publicado: {"humedad": 24.7}
Publicado: {"humedad": 25.0}
Publicado: {"humedad": 97.1}
Publicado: {"humedad": 97.2}
Publicado: {"humedad": 97.8}
Publicado: {"humedad": 97.8}

MicroPython (Raspberry Pi Pico) • Board CDC @ COM6
```

Posteriormente se comprobó el funcionamiento del broker MQTT. En la consola puede observarse cómo la Raspberry Pi Pico W establece la conexión correctamente y publica los mensajes que posteriormente serán consumidos por el servidor.

**Figura 3.** Recepción de los mensajes publicados por la Raspberry Pi Pico W en el broker Mosquitto.

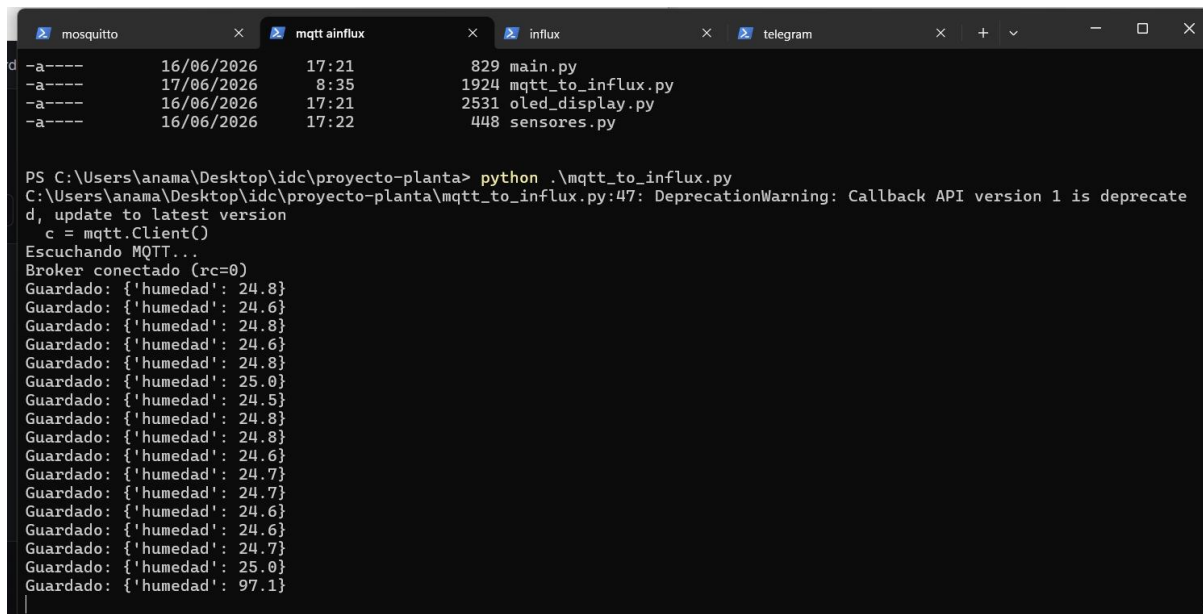


The screenshot shows a terminal window with multiple tabs. The active tab is `mosquitto`, which displays the output of the `mosquitto -c "C:\Program Files\mosquitto\mosquitto.conf" -v` command. The logs show the broker starting, loading the configuration, and listening on port 1883. It then shows two clients connecting: `auto-40F7EC2A-8CFD-FB64-8838-FC0A18029A68` and `pico1`. The `auto` client publishes a message to the `planta/sensores` topic, and the `pico1` client also publishes a message to the same topic. The logs show the details of these publications, including the topic, payload, and QoS.

```
mosquitto x mqtt influx x influx x telegram x
PS C:\Users\anama> mosquitto -c "C:\Program Files\mosquitto\mosquitto.conf" -v
1781690836: mosquitto version 2.1.2 starting
1781690836: Config loaded from C:\Program Files\mosquitto\mosquitto.conf.
1781690836: Bridge support available.
1781690836: Persistence support available.
1781690836: TLS support available.
1781690836: TLS-PSK support available.
1781690836: Websockets support available.
1781690836: Opening ipv4 listen socket on port 1883.
1781690836: mosquitto version 2.1.2 running
1781690929: New connection from 127.0.0.1:62866 on port 1883.
1781690929: New client connected from 127.0.0.1:62866 as auto-40F7EC2A-8CFD-FB64-8838-FC0A18029A68 (p4, c1, k60).
1781690929: No will message specified.
1781690929: Sending CONNACK to auto-40F7EC2A-8CFD-FB64-8838-FC0A18029A68 (0, 0)
1781690929: Received SUBSCRIBE from auto-40F7EC2A-8CFD-FB64-8838-FC0A18029A68
1781690929:   planta/sensores (QoS 0)
1781690929: auto-40F7EC2A-8CFD-FB64-8838-FC0A18029A68 0 planta/sensores
1781690929: Sending SUBACK to auto-40F7EC2A-8CFD-FB64-8838-FC0A18029A68
1781690935: New connection from 172.20.10.2:56277 on port 1883.
1781690935: New client connected from 172.20.10.2:56277 as pico1 (p4, c1, k0).
1781690935: No will message specified.
1781690935: Sending CONNACK to pico1 (0, 0)
1781690935: Received PUBLISH from pico1 (d0, q0, r0, m0, 'planta/sensores', ... (17 bytes))
1781690935: Sending PUBLISH to auto-40F7EC2A-8CFD-FB64-8838-FC0A18029A68 (d0, q0, r0, m0, 'planta/sensores', ... (17 bytes))
1781690965: Received PUBLISH from pico1 (d0, q0, r0, m0, 'planta/sensores', ... (17 bytes))
1781690965: Sending PUBLISH to auto-40F7EC2A-8CFD-FB64-8838-FC0A18029A68 (d0, q0, r0, m0, 'planta/sensores', ... (17 bytes))
1781690987: Received PINGREQ from auto-40F7EC2A-8CFD-FB64-8838-FC0A18029A68
1781690987: Sending PINGRESP to auto-40F7EC2A-8CFD-FB64-8838-FC0A18029A68
```

Una vez recibidos los mensajes, el script encargado de actuar como puente entre MQTT e InfluxDB procesa automáticamente cada medida y la almacena en la base de datos. Durante las pruebas se verificó que todas las publicaciones eran registradas correctamente.

**Figura 4.** Ejecución del script `mqtt_to_influx.py` almacenando las medidas recibidas.

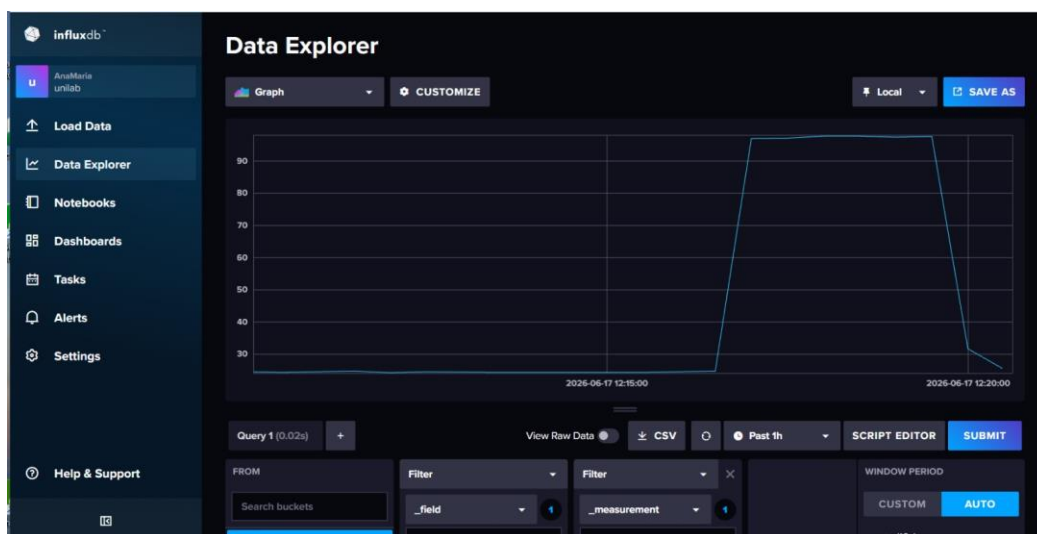


```
mosquitto x mqtt influx x telegram x + - □ x
d -a---- 16/06/2026 17:21 829 main.py
-a---- 17/06/2026 8:35 1924 mqtt_to_influx.py
-a---- 16/06/2026 17:21 2531 oled_display.py
-a---- 16/06/2026 17:22 448 sensores.py

PS C:\Users\anama\Desktop\idc\proyecto-planta> python .\mqtt_to_influx.py
C:\Users\anama\Desktop\idc\proyecto-planta\mqtt_to_influx.py:47: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
  c = mqtt.Client()
Escuchando MQTT...
Broker conectado (rc=0)
Guardado: {'humedad': 24.8}
Guardado: {'humedad': 24.6}
Guardado: {'humedad': 24.8}
Guardado: {'humedad': 24.6}
Guardado: {'humedad': 24.8}
Guardado: {'humedad': 25.0}
Guardado: {'humedad': 24.5}
Guardado: {'humedad': 24.8}
Guardado: {'humedad': 24.8}
Guardado: {'humedad': 24.6}
Guardado: {'humedad': 24.7}
Guardado: {'humedad': 24.7}
Guardado: {'humedad': 24.6}
Guardado: {'humedad': 24.6}
Guardado: {'humedad': 24.7}
Guardado: {'humedad': 25.0}
Guardado: {'humedad': 97.1}
```

A continuación se verificó el funcionamiento de InfluxDB. Las medidas almacenadas pudieron consultarse correctamente mediante el explorador de datos, comprobando que la información quedaba registrada de forma persistente.

**Figura 5.** Visualización de las medidas almacenadas en InfluxDB.

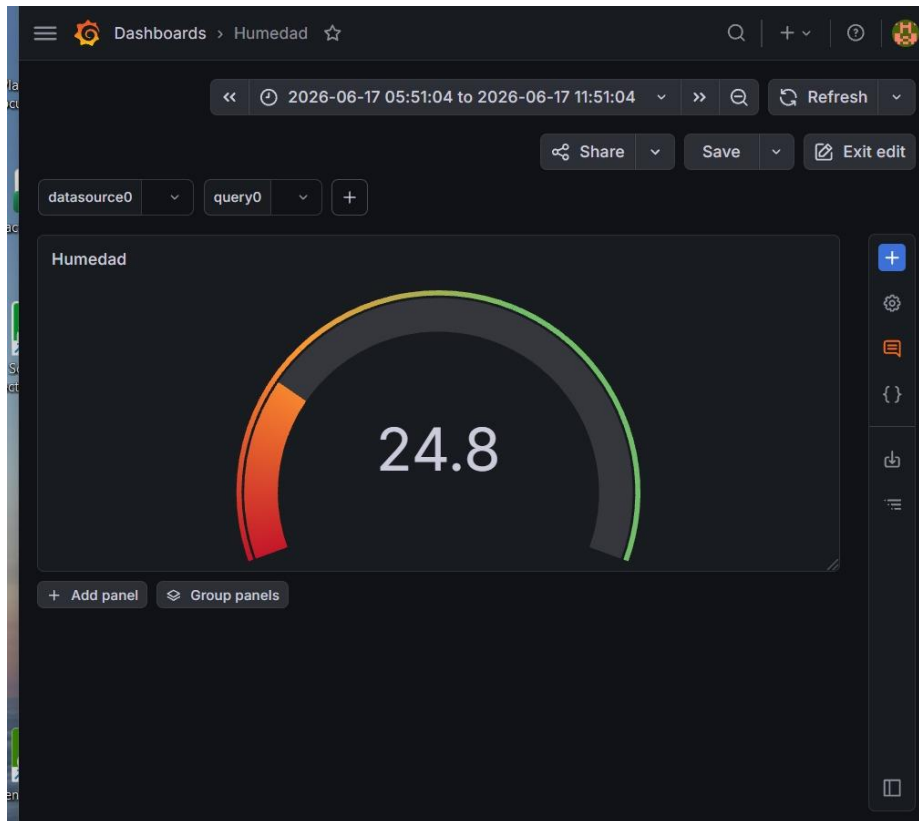


También se comprobó el correcto funcionamiento del dashboard desarrollado en Grafana. La información almacenada en InfluxDB se representa tanto mediante un



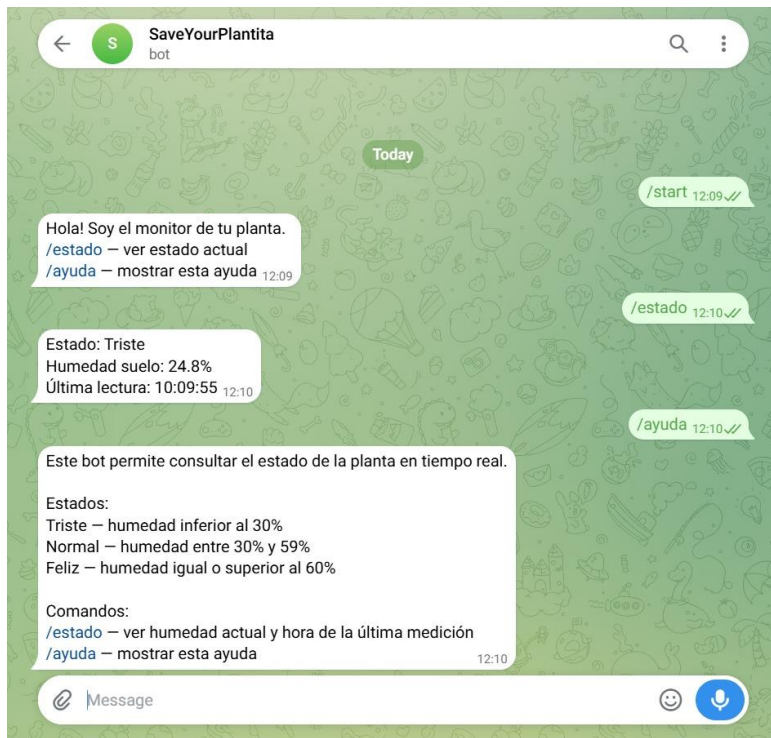
gráfico temporal como mediante un indicador tipo *gauge*, permitiendo conocer de forma rápida el nivel actual de humedad del suelo.

**Figura 6.** Dashboard desarrollado en Grafana para la monitorización de la humedad del suelo.



Finalmente se verificó el funcionamiento del bot de Telegram. El usuario puede consultar en cualquier momento el estado de la planta mediante comandos sencillos, obteniendo la humedad actual y la hora de la última medición registrada.

**Figura 7.** Consulta del estado de la planta mediante el bot de Telegram.



Durante las pruebas también se comprobó el correcto funcionamiento de los distintos servicios que forman parte del sistema. Tanto InfluxDB como el bot de Telegram permanecieron en ejecución de forma estable durante las sesiones de prueba, permitiendo el almacenamiento continuo de datos y la consulta remota del estado de la planta.

**Figura 8.** Ejecución del servidor InfluxDB.

```

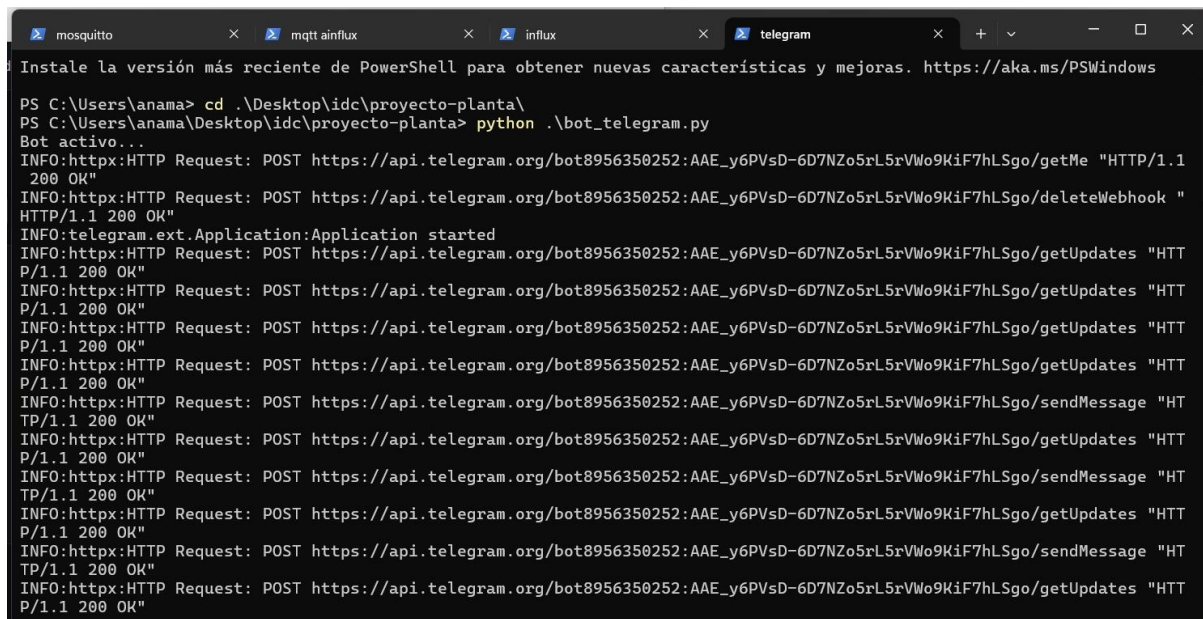
mosquitto x  mqtt ainflux x  influx x  telegram x
Directorio: C:\Users\anama\Downloads\influxdb2-2.9.1-windows_amd64

Mode                LastWriteTime         Length Name
----                -
-a----             16/06/2026    22:25      131381760 influxd.exe
-a----             16/06/2026    22:25         1067 LICENSE
-a----             16/06/2026    22:25         3355 README.md

PS C:\Users\anama\Downloads\influxdb2-2.9.1-windows_amd64> .\influxd
2026-06-17T10:07:52.850020Z    info    Welcome to InfluxDB    {"log_id": "13XGf03l000", "version": "v2.9.1", "commit": "d4fa1941fd", "build_date": "2026-05-11T20:49:02Z", "log_level": "info"}
2026-06-17T10:07:52.852601Z    info    Resources opened    {"log_id": "13XGf03l000", "service": "bolt", "path": "C:\Users\anama\influxdbv2\influxd.bolt"}
2026-06-17T10:07:52.852601Z    info    Resources opened    {"log_id": "13XGf03l000", "service": "sqlite", "path": "C:\Users\anama\influxdbv2\influxd.sqlite"}
2026-06-17T10:07:52.856687Z    info    Creating authorization store    {"log_id": "13XGf03l000", "store": "auth", "UseHashedTokens": true}
2026-06-17T10:07:52.858195Z    info    Checking InfluxDB metadata for prior version.    {"log_id": "13XGf03l000", "bolt_path": "C:\Users\anama\influxdbv2\influxd.bolt"}
2026-06-17T10:07:52.858195Z    info    Using data dir    {"log_id": "13XGf03l000", "service": "storage-engine", "service": "store", "path": "C:\Users\anama\influxdbv2\engine\data"}
2026-06-17T10:07:52.858195Z    info    Compaction settings    {"log_id": "13XGf03l000", "service": "storage-engine", "service": "store", "max_concurrent_compactions": 8, "throughput_bytes_per_second": 50331648, "throughput_bytes_per_second_burst": 50331648}
2026-06-17T10:07:52.858195Z    info    Open store (start)    {"log_id": "13XGf03l000", "service": "storage-engine", "service": "store", "op_name": "tsdb_open", "op_event": "start"}

```

**Figura 9.** Ejecución del bot de Telegram en el servidor.



```
mosquitto x mqtt ainflux x influx x telegram x + - □ ×
Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\anama> cd .\Desktop\idc\proyecto-planta\
PS C:\Users\anama\Desktop\idc\proyecto-planta> python .\bot_telegram.py
Bot activo...
INFO:httpx:HTTP Request: POST https://api.telegram.org/bot8956350252:AAE_y6PVsD-6D7NZo5rL5rVWo9KiF7hLSgo/getMe "HTTP/1.1 200 OK"
INFO:httpx:HTTP Request: POST https://api.telegram.org/bot8956350252:AAE_y6PVsD-6D7NZo5rL5rVWo9KiF7hLSgo/deleteWebhook "HTTP/1.1 200 OK"
INFO:telegram.ext.Application started
INFO:httpx:HTTP Request: POST https://api.telegram.org/bot8956350252:AAE_y6PVsD-6D7NZo5rL5rVWo9KiF7hLSgo/getUpdates "HTTP/1.1 200 OK"
INFO:httpx:HTTP Request: POST https://api.telegram.org/bot8956350252:AAE_y6PVsD-6D7NZo5rL5rVWo9KiF7hLSgo/getUpdates "HTTP/1.1 200 OK"
INFO:httpx:HTTP Request: POST https://api.telegram.org/bot8956350252:AAE_y6PVsD-6D7NZo5rL5rVWo9KiF7hLSgo/getUpdates "HTTP/1.1 200 OK"
INFO:httpx:HTTP Request: POST https://api.telegram.org/bot8956350252:AAE_y6PVsD-6D7NZo5rL5rVWo9KiF7hLSgo/getUpdates "HTTP/1.1 200 OK"
INFO:httpx:HTTP Request: POST https://api.telegram.org/bot8956350252:AAE_y6PVsD-6D7NZo5rL5rVWo9KiF7hLSgo/sendMessage "HTTP/1.1 200 OK"
INFO:httpx:HTTP Request: POST https://api.telegram.org/bot8956350252:AAE_y6PVsD-6D7NZo5rL5rVWo9KiF7hLSgo/getUpdates "HTTP/1.1 200 OK"
INFO:httpx:HTTP Request: POST https://api.telegram.org/bot8956350252:AAE_y6PVsD-6D7NZo5rL5rVWo9KiF7hLSgo/sendMessage "HTTP/1.1 200 OK"
INFO:httpx:HTTP Request: POST https://api.telegram.org/bot8956350252:AAE_y6PVsD-6D7NZo5rL5rVWo9KiF7hLSgo/getUpdates "HTTP/1.1 200 OK"
INFO:httpx:HTTP Request: POST https://api.telegram.org/bot8956350252:AAE_y6PVsD-6D7NZo5rL5rVWo9KiF7hLSgo/sendMessage "HTTP/1.1 200 OK"
INFO:httpx:HTTP Request: POST https://api.telegram.org/bot8956350252:AAE_y6PVsD-6D7NZo5rL5rVWo9KiF7hLSgo/getUpdates "HTTP/1.1 200 OK"
```

En conjunto, los resultados obtenidos demuestran que todos los componentes del sistema funcionan de forma integrada. La Raspberry Pi Pico W adquiere correctamente las medidas del sensor de humedad, las publica mediante MQTT, el servidor las almacena en InfluxDB y posteriormente pueden visualizarse en Grafana o consultarse remotamente mediante Telegram. De esta forma se cumplen los objetivos planteados al inicio del proyecto, obteniendo un sistema IoT funcional para la monitorización del estado de una planta.

## PROBLEMAS ENCONTRADOS Y SU RESOLUCIÓN

Durante el desarrollo del proyecto surgieron diversos problemas técnicos que requirieron modificaciones en el diseño inicial y la adopción de soluciones específicas.

### Uso de imágenes en la pantalla TFT

Inicialmente se planteó utilizar imágenes en formato PNG o JPEG directamente en la Raspberry Pi Pico W. Sin embargo, la pantalla ST7735S requiere trabajar con formatos adaptados al hardware y el procesamiento de imágenes comprimidas supone un consumo elevado de memoria.

Como solución, las imágenes fueron convertidas previamente a formato RAW RGB565 mediante una herramienta de conversión desarrollada para el proyecto. De esta forma la Raspberry Pi Pico W puede cargar y mostrar las imágenes de forma eficiente.

### Imagen del Tamagotchi no ocupaba toda la pantalla

Uno de los problemas encontrados fue que las imágenes del Tamagotchi no se mostraban ocupando toda la pantalla TFT ST7735S. Esto se debía a que las imágenes

originales no tenían exactamente la misma resolución que la pantalla, que es de 128x160 píxeles.

Para solucionarlo, se redimensionaron las imágenes al tamaño correcto antes de convertirlas al formato RAW RGB565. De esta forma, cada imagen generada ocupa toda la superficie disponible de la pantalla y se muestra correctamente en orientación vertical.

### Integración de los distintos componentes software

El proyecto combina múltiples tecnologías diferentes: MicroPython, MQTT, InfluxDB, Grafana y Telegram. La correcta comunicación entre todos los componentes fue uno de los principales retos del desarrollo.

Para facilitar la depuración se realizaron pruebas independientes de cada módulo antes de integrarlos en el sistema completo, verificando progresivamente la comunicación entre los distintos elementos.

### Configuración del broker MQTT

Durante las pruebas de comunicación se comprobó que la Raspberry Pi Pico W necesitaba conectarse a la dirección IP real del ordenador donde se ejecutaba Mosquitto, no a localhost.

La solución consistió en obtener la IP local del PC y configurarla correctamente en el archivo config.py como dirección del broker MQTT. Además, se verificó que la Pico W y el ordenador estuvieran conectados a la misma red WiFi.

## CONCLUSIONES

Durante el desarrollo de este proyecto se ha diseñado e implementado un sistema IoT capaz de monitorizar el estado de una planta mediante la medición de la humedad del suelo. El sistema integra correctamente todos los componentes hardware y software necesarios para realizar la adquisición de datos, su transmisión mediante MQTT, el almacenamiento en InfluxDB y su posterior visualización mediante Grafana.

Además, se ha desarrollado una interfaz gráfica basada en un personaje tipo Tamagotchi que permite conocer de forma intuitiva el estado de la planta, así como un bot de Telegram que facilita la consulta remota de la última medición registrada.

El proyecto ha permitido aplicar de forma práctica los conocimientos adquiridos durante la asignatura, especialmente en el uso de sensores, programación en MicroPython, protocolos de comunicación IoT, bases de datos temporales y herramientas de monitorización.

En cuanto a posibles mejoras futuras, el sistema podría ampliarse incorporando nuevos sensores, como temperatura, luminosidad o humedad ambiental, permitiendo obtener una monitorización más completa de la planta. También sería posible implementar notificaciones automáticas cuando la humedad descienda por debajo de un determinado umbral o ampliar el sistema para gestionar varias plantas simultáneamente.

En definitiva, se han alcanzado los objetivos planteados al inicio del proyecto, obteniendo un sistema funcional, modular y fácilmente ampliable, capaz de monitorizar el estado de una planta y ofrecer la información tanto de forma local como remota.